



Universidad autónoma de baja California

Ingeniería en computación

Ingeniería en software

Practica 6.1: scripts y alias

Alma Leticia Palacios Guerreo

Garcia Chávez Erik

Viernes 28 de septiembre del 2025

Scripts y alias

Comando alias:

En ocasiones se suelen utilizar comandos que son difíciles de teclear o de recordar. UNIX ofrece la posibilidad de dar un alias o nombre corto a un comando con o sin parámetros, para que cada vez que se quiera ejecutar, sólo se escriba el alias.

Sintaxis:

Alias <nombre_alias>="<comando>"

Ejemplo: **\$alias lista="ls -l"**

Comnado unalias:

Este comando funciona para borrar un alias creado dentro de nuestra sesión de trabajo.

Sintaxis

Unalias <nombre_alias>

Ejemplo: **unalias lista**

Programación de Shell.

Shell Script. Un shell script es un archivo de texto ordinario que contiene una colección de uno o más comandos del shell.

Ventajas del uso de shell scripts

- Evita escribir secuencia de comandos que son largas o complicadas.
- Permite automatizar tareas.
- Evita errores.

Reglas para escribir un shell script

- Use sangrías para separar las líneas del script, así será más fácil de leer.
- El enter se considera el fin de comando.
- Para continuar un comando en la línea siguiente escriba una diagonal inversa (\) al final de la línea.
- Los espacios, tabuladores y líneas en blanco son irrelevantes.
- Para agregar comentarios a sus scripts simplemente escriba # al inicio de la línea

Pasos para la creación de un shell script

1. Generar el archivo de texto con un editor de texto como vi, nano, pico, emacs, etc.
2. Modificar los permisos para hacer el archivo ejecutable.
3. Ejecute el script con \$./

Comandos del Shell

- **Clear** . limpiar la pantalla
- **echo**. Sirve para enviar datos a la salida estándar, estos datos pueden ser cadenas o valores de variables. Automáticamente agrega el salto de línea, a menos que se especifique la opción -n.
 - o **echo "Hola"** Envía a la pantalla la palabra Hola.
 - o **echo \$saludo** Muestra en pantalla el contenido de la variable saludo.
 - o **echo "Hola /c"** Envía el mensaje a la pantalla, pero no hace salto de línea.
- **read**. Este comando lee variables desde el teclado. El delimitador es el espacio. Si el usuario escribe más de una palabra se asigna a la primera variable y el resto a la segunda. Generalmente se usa acompañado de echo ya que es necesario enviar un mensaje (echo) para que indique al usuario que se necesita un dato de entrada. Todos los datos que se leen son cadenas de caracteres aun los números
 - o **read varibale** -> lee una varibale
 - o **read varibales1 varibale 2** -> lee dos varibales

ejemplos:

- o **echo "Como te llamas?"**
read nombre echo
- o **"Cual es tu nombre y matrícula?"**
read nombre matricula

Asignación directa de valores a variables

Para darle un valor a una variable simplemente se escribe el nombre de la variable sin espacios, seguido del signo de igual y enseguida el valor. La sintaxis es la siguiente:

nombre-variable=valor

Nota: Cuando el valor de la variable es más de una palabra debe ir entrecomillado.

Ejemplos:

Ciudad: Tijuana

domicilio= "Calzada Tecnológico"

edad=26

Variables Posicionales (argumentos de la línea de comando)

Cuando al ejecutar un comando se indican parámetros, estos se almacenan en un arreglo de caracteres el cual se le llama **variables posicionales**. La variable 0 es el nombre del programa, la 1 corresponde al primer argumento, la 2 al segundo y así sucesivamente. En la siguiente línea de comando

\$ agenda amigo 5674 Tecate

La variable posicional 0 contiene el nombre del script: **agenda**

La variable posicional 1 contiene el primer parámetro escrito: **amigo**

La variable posicional 2 contiene el segundo parámetro escrito: **5674**

La variable posicional 3 contiene el tercer parámetro escrito: **Tecate**

Acceso a las variables posicionales.

echo "El script en ejecución es \$0"

echo "Hola \$1"

Variables Predeterminadas

\$# :num. de argumentos

\$* :todos los arg de la shell

\$\$:PID de la Shell

\$HOME

\$PATH

Sustitución de comando

La comilla invertida ` indica al shell que la cadena entrecomillada es un comando a ejecutarse, por ejemplo, en el siguiente script la variable fecha toma el valor que resulta de ejecutar el comando date, lo que se conoce como sustitución de comando.

Ejemplo de sustitución de comando.

clear

actual=`pwd`

echo " El directorio actual es \$actual"

Caracteres especiales dentro de los scripts

Los siguientes símbolos son considerados caracteres especiales dentro de los scripts para poderlos usarlos dentro en el comando echo, es necesario anteponerles la diagonal invertida o backslash [\].

- \$ Hace referencia al contenido de una variable.
- | Entubamiento.
- # Comentario
- & Ejecución de un proceso en el background.
- ? Comodín para un solo caracter.
- * Comodín para varios caracteres.
- > Redireccionamiento de salida.
- < Redireccionamiento de entrada.
- ` Sustitución de comando.
- >> Redireccionamiento de salida.
- [] Rango de caracteres.
- [a-z] Caracteres de a a z.

Aspectos básicos del lenguaje bash

Bloques selectivos *if-elseif-else* , [], test.

Comandó *test* o []

Los corchetes o **test** se usan para probar expresiones. Comúnmente usadas en los bloques selectivos de if-elseif-else

Con []

if [expresión]; then

operaciones

fi

Con test

```
if test expresión; then
    operaciones
fi
```

Se ejecutará una acción si la expresión de prueba es verdadera. Si no se ejecutarán las operaciones de la parte falsa (else).

Con []

```
if [ expresión ]; then
    operaciones
else
    operaciones
fi
```

Con test

```
if test expresión; then
    operaciones
else
    operaciones
fi
```

La siguiente estructura del if permite anidar estructuras if, para permitir la selección de múltiples alternativas. Utiliza el comando elif, el cual prueba la variable de nuevo si el primer if no fue verdadero. Si ninguna de las pruebas es verdadero, entonces se ejecuta la parte falsa del elif.

```
if [ expresión ]; then
    operaciones
elseif [ expresion ]; then
    operaciones
else
    operaciones
fi
```

Operadores Lógicos

! Not

-a And

-o Or

Operadores de archivos

-d archivo Verdadero si el archivo es un directorio.

-e archivo Verdadero si el archivo existe.

-f archivo Verdadero si el archivo normal.

-r archivo Verdadero si el archivo se puede leer.

-s archivo Verdadero si el archivo tiene longitud mayor de 0 bytes.

-w archivo Verdadero si el archivo se puede escribir.

-x archivo Verdadero si el archivo se puede ejecutar.

1- Cree un alias para los comandos date, cp y finger.

```
garcia07@limesurvey ~$ alias date="fecha"
garcia07@limesurvey ~$ alias cp="copiar"
garcia07@limesurvey ~$ finger
Login      Name            Tty      Idle    Login Time   Office       Office Phone
garcia07   pts/2           pts/2    Nov  4 00:11 (10.32.44.51)
jasonh     pts/4           pts/4    Nov  3 22:11 (10.32.75.170)
karey      pts/9           pts/9    Nov  3 23:45 (10.32.106.196)
kvaldez    pts/3           pts/3    Nov  3 21:51 (10.32.14.29)
lbryan     pts/7           pts/7    Nov  3 23:16 (10.32.18.23)
morales84  pts/8           pts/8    Nov  3 23:22 (10.32.7.82)
root       Leticia Palacios pts/0     2d Oct 31 04:23 (148.231.169.208)
sharai14   pts/1           pts/1    Nov  3 22:54 (10.32.209.109)
garcia07@limesurvey ~$ alias finger="quien_esta"
```

```
alias cp='copiar'
alias date='fecha'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias finger='quien_esta'
```

la forma de crear el alias es mediante el comando **alias** entonces se ejecuta

alias date =" fecha" para indicar que fecha es lo mismo que el comando **date** lo mismo lo agrego para los otros

2- Hacer un shell script que muestre solamente la hora del sistema.

```
$ date
Tue Nov  4 00:35:24 UTC 2025
$ hora_sistem
-sh: 51: hora_sistem: not found
$ ./hora_sistem
hora actual es: 00:35:32
$ |
```

```
#!/bin/bash
#mostrar solamente la fecha actual

hora=`date | cut -d" " -f5`

echo "hora actual es: $hora"
```

Aquí lo que se está haciendo es una sustitución de comandos, entonces ahora **hora** tiene 2 comandos que realizarán que el script solo devuelva la hora que tiene el sistema

El comando primero nos da la hora, por medio de un entubamiento tomamos esa salida y la cortamos, le indicamos que queremos solo la columna 5 que es en donde esta la hora y así es como lo obtenemos y imprimimos la variable.

3- . Hacer un shell script que muestre el directorio actual y el nombre real del usuario.

```
$ chmod 777 pwd_who
$ ./pwd_who
hola garcia07 estas en la direccion /home/sistop252/garcia07/practica6_alias_shell
$ |
```

```
#!/bin/bash

directorio=`pwd`
quien=`whoami`

echo "hola $quien estas en la direccion $directorio"
```

Realizamos la sustitución de comandos, para guardar el comando en una variable y solo se imprime, el comando **pwd** nos da el directorio actual en donde se esta trabajando y el comando **whoami** indica el nombre del usuario.

4- Hacer un shell script que pida el nombre de un archivo, busque el archivo y muestre su contenido, el número de líneas, palabras y caracteres que contiene

```
#!/bin/bash

if [ -e $1 ]; then
    #el archivo por lo menos existe
    if [ -r $1 ]; then
        mostrar=`cat $1`
        mostrar_c=`wc -c <$1`
        mostrar_w=`wc -w <$1`
        mostrar_l=`wc -l <$1`

        echo "contenido -->$mostrar"
        echo "lineas: $mostrar_l"
        echo "palabras: $mostrar_w"
        echo "caracteres: $mostrar_c"
    else
        echo "cambia los permisos de lectura para el archivo"
    fi
else
    echo "el archivo con el nombre $1 no existe en el directorio"
fi
```

```
$ ls
hora_sistem  hormigas  pwd_who  show_item
$ nano show_item
$ ./show_item hormigas
contenido -->Cuentan que una colonia de hormigas decidió un día salir a excursión. Iban en busca de un nuevo lugar donde pudieran crear su nido y sentar las bases de la nueva colonia. Un lugar al que el mundo recuerda como Utopia.
```

```
lineas: 37
palabras: 712
caracteres: 4196
```

el script recibirá como variable de línea de comando el archivo con el cual quiere mostrar la información, estas variables se representan en bash como \$0 que representa el comando \$1 el primero parámetro, \$2 el segundo, etc.

Por lo que primero verifica con el operador de archivo **-e** que el documento exista en el directorio actual, en caso de que si exista debe de tener los permisos habilitados de lectura **-r** para que el script pueda funcionar de manera eficiente.

Realizamos la sustitución de comando del comando **wc** el cual con sus diferentes parámetros **-l ->** nos muestra el total de líneas que tiene el archivo, **-c ->** nos muestra el total de caracteres que tiene, **-w ->** nos muestra el total de palabras que tiene el archivo

- 5- . Hacer un shell script que pida los nombres tres archivos, sustituya los caracteres de mayúsculas a minúsculas en el primero, cambie todos los números por asteriscos al segundo y el tercero permanece igual. Muestre en pantalla los tres archivos y pida al usuario el nombre del archivo donde se guardarán los tres archivos.

```
#!/bin/sh

echo "ingrese el nombre del primer archivo"
read archivo1

echo "ingrese el nombre del segundo archivo"
read archivo2

echo "ingrese el nombre del tercer archivo"
read archivo3

#es importante es usar las comillas para la comparacion de otro modo siempre devolvera verdadero,
if [ -e "$archivo1" ] && [ -e "$archivo2" ] && [ -e "$archivo3" ]; then
    lower=`tr "A-Z" "a-z" <$archivo1`
    number_car=`tr "[0-9]" "*" <$archivo2`
    imprimir=`cat $archivo3`

    echo "archivo 1 mayusculas -> minusculas $lower"
    echo "archivo 2 numeros -> asteriscos $number_car"
    echo "el archivo 3 permanece igual $imprimir"

else
    #aqui lo que estoy haceindo es como mapear que archivo es el que no existe, ene el caso que un ar
    chivo no exista
    #la condicion me dara verdadera porqie marcara como falsa pero al negarla sera verdadera
    #y los archivos verdaeros que si estan seran faltos por lo que no entraran
    if [ ! -e "$archivo1" ]; then
        echo "el archivo $archivo1 no existe en la ruta"

    elif [ ! -e "$archivo2" ]; then
        echo "el archivo $archivo2 no existe en la ruta"

    else
        echo "el archivo $archivo3 no existe en la ruta"
    fi
fi
```

Salida:

```
normtgas_c
archivo 1 mayusculas -> minusculas      11      lopez flores jose juan      1231769
    12 lopez moreno nereo cesar      1247158
    13 lopez trujillo ricardo donovan      1239191
    16 robledo lopez alan omar      1239405
este texto sustituye al anterior
esta linea se agrega al final
jan joshua
jan ross
dec raul
apr rocha
dec alane
oct alejandro
jun hkboy
jan stivy
lllllll
l
lllll
lllllllllllll
lllxlllxlllxllll,llllzl
llll
lllllll
```

```

archivo 2 numeros -> asteriscos BANDERAS GAMBOA JULIO CESAR
CASTRO FLORES Noemi *****
CRESCENCIO PALACIOS ALI *****
CRISOSTOMO SUAREZ CARLOS ERNESTO *****
CRUZ TAFOYA JESUS *****
DE LEON RANGEL SERGIO ALEJANDRO *****
ESTRADA MOLINA WILIBER DANIEL *****
GONZALEZ PEREZ DAVID *****
GUTIERREZ LEON RICARDO *****
HERRERA GONZALEZ LIAM *****
MARINA ANAYA BRAYAN MANUEL *****
MENDOZA COLIN NOEMI *****
OROZCO AMADOR AXEL AARON *****
PEINADO MORENO JAVIER ALEXIS *****
PEREZ FRANCO CESAR OSCAR *****
PEREZ VILLA JAIME ALBERTO *****
PUENTE MORENO LILIAN ALEJANDRA ***** Noemi
SALTO MALFAVON JESUS VICENTE *****
SUAREZ GONZALEZ CARLOS *****
VAZQUEZ BELTRAN IVAN *****
ZEPEDA LOPEZ GUILLERMO *****
el archivo 3 permanece igual
las hormigas son muy feliz

```

Archivo original

```

$ cat COMBO
  11 LOPEZ FLORES JOSE JUAN      1231769
  12 LOPEZ MORENO NEREO CESAR   1247158
  13 LOPEZ TRUJILLO RICARDO DONOVAN 1239191
  16 ROBLEDO LOPEZ ALAN OMAR    1239405
ESTE TEXTO SUSTITUYE AL ANTERIOR
ESTA LINEA SE AGREGA AL FINAL
JAN JOSHUA
JAN ROSS
DEC RAUL
APR ROCHA
DEC ALANE

```

Archivo 2 original

```

$ cat lista
BANDERAS GAMBOA JULIO CESAR      1260387
CASTRO FLORES Noemi              1256631
CRESCENCIO PALACIOS ALI          1254871
CRISOSTOMO SUAREZ CARLOS ERNESTO 1253034
CRUZ TAFOYA JESUS                1254746
DE LEON RANGEL SERGIO ALEJANDRO   1254187
ESTRADA MOLINA WILIBER DANIEL     1255547
GONZALEZ PEREZ DAVID              1257377
GUTIERREZ LEON RICARDO            1253624
HERRERA GONZALEZ LIAM             1251071
MARINA ANAYA BRAYAN MANUEL        1256051
MENDOZA COLIN NOEMI              1249857
OROZCO AMADOR AXEL AARON          1253590
PEINADO MORENO JAVIER ALEXIS      1245864
PEREZ FRANCO CESAR OSCAR          1216707
PEREZ VILLA JAIME ALBERTO         1255288
PUENTE MORENO LILIAN ALEJANDRA    1244911 Noemi
SALTO MALFAVON JESUS VICENTE      1249597
SUAREZ GONZALEZ CARLOS            1252642
VAZQUEZ BELTRAN IVAN             1247545
ZEPEDA LOPEZ GUILLERMO           1252186

```

Salida del archivo 3

```
$ cat hormigas_c
las hormigas son muy feliz
$ |
```

para realizar el programa primero pedimos al usuario el nombre de los 3 archivos, mediante **echo** y **read**, importante que los archivos que se le van a solicitar estén en el directorio actual.

Antes de seguir con debemos de verificar que en efecto los archivos existen, para eso hacemos uso de las **condiciones** y **operadores lógicos** para garantizar que esos archivos existan:

```
#es importante es usar las comillas para la comparación de otro modo stem
if [ -e "$archivo1" ] && [ -e "$archivo2" ] && [ -e "$archivo3" ]; then
```

Si no es el caso manda al else, lo que hace el else es ver que archivo es el que falla, negamos la condicional **-e** este operador de archivo me devuelve verdadero si el archivo existe, entonces negamos porque si el archivo existe no nos interesa por lo que lo volverá falso y no entrara a la condición, en cambio si no existe devolverá falso y con la negación será verdadero y me dirá X archivo es el que no existe.

```
else
    #aqui lo que estoy haciendo es como mapear que archivo es el
    #la condicion me dara verdadera porqie marcara como falsa p
    #y los archivos verdaeros que si estan seran faltos por lo
    if [ ! -e "$archivo1" ]; then
        echo "el archivo $archivo1 no existe en la ruta"

    elif [ ! -e "$archivo2" ]; then
        echo "el archivo $archivo2 no existe en la ruta"

    else
        echo "el archivo $archivo3 no existe en la ruta"
    fi
fi
```

Ejemplo:

Ingrese 2 archivos que si existen en mi directorio actual **upper_lower** y **números**, pero **perros** no existe y me indica que ese archivo es el que no existe

```
└─$ ./question_5
ingrese el nombre del primer archivo
upper_lower
ingrese el nombre del segundo archivo
numeros
ingrese el nombre del tercer archivo
perros
el archivo perros no existe en la ruta

└─(dmrx dmr)-[~/S0]
└─$ ls
ejemplo_chmod  hormigas  numeros  queestion_10  question_5  question_6  question_8  question_9  upper_lower
```

En el caso que, si existan los archivos, lo que hago es una sustitución de comandos, y uso el comando `tr` para remplazar del primero archivo todas las mayúsculas a minúsculas, el segundo todos los números por `*` y el tercero se queda igual, solo lo imprimó

```
lower=`tr "[A-Z]" "[a-z]" <$archivo1`  
number_car=`tr "[0-9]" "*" <$archivo2`  
imprimir=`cat $archivo3`  
  
echo "archivo 1 mayusculas -> minusculas $lower"  
echo "archivo 2 numeros -> asteriscos $number_car"  
echo "el archivo 3 permanece igual $imprimir"
```

- 6- Hacer un shell que pida el nombre real de una persona, encuentre su nombre de usuario, muestre los procesos que está ejecutando y le envíe un mensaje en tiempo real.

```
#!/bin/bash  
  
echo "ingrese el nombre real (nombre de usuario): "  
read nombre  
  
user_name=`w | grep "$nombre" | cut -d" " -f1`  
  
if [ -z "$user_name" ]; then  
    echo "usuario no encontrado encontrado"  
    exit 1  
fi  
  
echo "el usaurio $user_name tiene los siguientes proceso"  
ps -u "$user_name"  
  
#echo $procesos  
  
echo "ingrese el mensaje a enviar:"  
read mensaje  
  
send=`"$mensaje" | write "$user_name"`
```

```

ingrese el nombre real (nombre de usuario):
karey
el usaurio karey tiene los siguientes proceso
      PID TTY          TIME CMD
  84652 ?                00:00:00 sshd
  84653 pts/15          00:00:00 sh
  86005 pts/15          00:00:00 nano
ingrese el mensaje a enviar:
hola

```

lo que se realizo es primero pedirle al usuario a quien quiere buscar en el servidor. Entonces la variable "user_name" es la que sustituye el comando

primero con **w** listamos todos los usuarios que están conectados,

después con **grep** tomamos esa salida y le indicamos que queremos buscar la coincidencia que tenga el nombre del usuario

por último cortamos el nombre el cual se encuentra en la primera columna del comando **w**

para poder seguir la variable debe de tener algo, nos devio de regresar el nombre dentro de los usuarios, en otro caso no lo encontró puesto que no esta conectado y sale del script

```

user_name=`w | grep "$nombre" | cut -d" " -f1`

if [ -z "$user_name" ]; then
    echo "usuario no encontrado encontrado"
    exit 1
fi

```

Si lo encontró entonces lista sus procesos y pido el mensaje que se quiere enviar.

```

echo "el usaurio $user_name tiene los siguientes proceso"
ps -u "$user_name"
#echo $procesos
echo "ingrese el mensaje a enviar:"
read mensaje
send=`"$mensaje" | write "$user_name"`

```

7- . Elimine el alias para el comando cp.

```
$ alias  
copiar='cp'  
$ unalias copiar  
$ |
```

8- Hacer un shell que tome de la línea de comando un número N y un nombre de archivo. Muestre las primeras y últimas N líneas del archivo especificado en la línea de comando.

```
#!/bin/bash  
  
echo "ingrese un numero:"  
read lineas_n  
  
echo "ingrese un archivo"  
read archivo  
  
#antes que nada deberiamos de verificar que existan estos docuemtnos  
if [ -e "$archivo" ]; then  
    echo "las primeras $lineas_n del archivo"  
  
    head -"$lineas_n" <$archivo  
  
    #echo $cabeza  
    echo "las ultimas $lineas_n del archivo"  
  
    tail -"$lineas_n" <$archivo  
  
else  
    echo "el archivo $archivo no existe prueba de nuevo"  
  
fi
```

Salida:

```
$ ./question_8  
ingrese un numero:  
5  
ingrese un archivo  
hormigas  
las primeras 5 del archivo  
Cuentan que una colonia de hormigas decidió un día salir a excursión. Iban en busca de un nuevo lugar donde pudieran crear su nido y sentar las bases de la nueva colonia. Un lugar al que el mundo recuerda como Utopia.  
Y dice la leyenda que las hormigas viajaban y viajaban, y el camino se volvía cada vez más penoso y peligroso.  
Algunas hormigas decidieron regresar, otras se quedaron varadas en el camino, pues carentes de ánimo pronto la debilidad las invadió y fueron incapaces de seguir adelante.  
las ultimas 5 del archivo  
Y a todas ellas las hormigas, les hablaban de que ya vivían en la tierra de. Y que el paraíso era ese y no otro. Todas las hormigas que escuchaban las tachaban de locas y decían que habían enfermado por haber permanecido tanto tiempo en la montaña.  
Pasaron los años y los siglos y los milenios, y el hombre sigue buscando aún una tierra de ensueño y magia a la que busca llegar, atravesando valles y subiendo montañas, salvando peligros y descubriendo misterios.  
Tal vez algún día logre subir tan alto que pueda hacer el mismo descubrimiento que nuestras amigas las hormigas.  
$ |
```

Se le pide al usuario 2 variables que serán **líneas_n** y **archivo** las cuales dictan el número de líneas que se quiere imprimir del archivo tanto del inicio como del final

Verificamos que el archivo exista en el directorio actual

```
if [ -e "$archivo" ]; then
```

usamos el comando **head** para imprimir las primeras **líneas_n** líneas del archivo y usamos el comando **tail** para imprimir las ultimas **líneas_n** líneas del archivo.

```
echo "las primeras $lineas_n del archivo"
head -"$lineas_n" <$archivo
#echo $cabeza
echo "las ultimas $lineas_n del archivo"
tail -"$lineas_n" <$archivo
```

- 9- Hacer un shell que lea el nombre de un archivo, enseguida pregunte los permisos que, sobre el archivo, desea otorgar para usuario, grupo y otros. Los permisos se pedirán en forma individual para cada entidad.

```
#!/bin/bash

echo "ingrese el nombre del archivo"
read archivo

#debemos de sacar el substring de cada uno para saber que permiso se le otorgan

if [ -e "$archivo" ]; then

    echo "ingrese los permisos para usuario (EJ:rwx): "
    read perm_user

    echo "ingrese los permisos para grupo (EJ: r-x):"
    read perm_group

    echo "ingrese los permisos para otros:"
    read perm_other

    user_ch=`echo "$perm_user" | tr -d '- '`
    group_ch=`echo "$perm_group" | tr -d '- '`
    other_ch=`echo "$perm_other" | tr -d '- '`

    permisos="u=$user_ch,g=$group_ch,o=$other_ch"

    chmod $permisos "$archivo"

    echo "permisos cambiados"

    ls -l

else
    echo "el archivo $archivo no existe"
fi
```

Salida:

```
$ touch archivo_ch
$ ls
COMBO      hora_sistem  hormigas_c    lista  question_6  question_9
archivo_ch hormigas     items_replace_5  pwd_who  question_8  show_item
$ ./question_9
ingrese el nombre del archivo
archivo_ch
ingrese los permisos para usuario (EJ: rwx):
rwx
ingrese los permisos para grupo (EJ: r-x):
r-x
ingrese los permisos para otros:
r--
permisos cambiados
total 48
-rw-r--r-- 1 garcia07 sistop252 501 Nov 5 02:56 COMBO
-rwxr-xr-- 1 garcia07 sistop252 0 Nov 5 03:55 archivo_ch
-rwxrwxrwx 1 garcia07 sistop252 109 Nov 4 00:35 hora_sistem
-rw-r--r-- 1 garcia07 sistop252 4196 Nov 4 01:10 hormigas
-rw-r--r-- 1 garcia07 sistop252 29 Nov 5 03:01 hormigas_c
-rwxr--r-- 1 garcia07 sistop252 1279 Nov 5 02:54 items_replace_5
-rw-r--r-- 1 garcia07 sistop252 1091 Nov 5 02:59 lista
-rwxrwxrwx 1 garcia07 sistop252 105 Nov 4 00:54 pwd_who
-rwxr--r-- 1 garcia07 sistop252 412 Nov 5 03:44 question_6
-rwxr--r-- 1 garcia07 sistop252 462 Nov 5 03:48 question_8
-rwxr--r-- 1 garcia07 sistop252 766 Nov 5 03:52 question_9
-rwxrwxrwx 1 garcia07 sistop252 438 Nov 4 01:44 show_item
$ |
```

Le pedimos al usuario el nombre del archivo, pero debemos de verificar que este archivo exista por lo que el operador **-e** devolverá verdadero si el archivo existe si es el caso entonces debemos de preguntar que permisos quiere ponerle al archivo cada uno esta separado por **permisos del usuario, permiso de grupo y otros**.

Por lo que se toma esos permisos el formato que se espera es **rwx** si un permiso quiere ser quitado le ponemos **-** en la posición en donde debería de ir el permiso

Como el usuario la forma de ingresar seria con **-** lo que hacemos es suprimir en donde se supone que quita los permisos, si se da el caso con el comando **tr -d** entonces queda limpio el formato para poderlo pasar al comando **chmod** que se encargara de cambiar los permisos del archivo

Por último, solo listo los archivos dentro del directorio mostrando los permisos con **ls -l**

```

echo "ingrese los permisos para usuario (EJ: rwx): "
read perm_user

echo "ingrese los permisos para grupo (EJ: r-x):"
read perm_group

echo "ingrese los permisos para otros:"
read perm_other

user_ch=`echo "$perm_user" | tr -d '-'`
group_ch=`echo "$perm_group" | tr -d '-'`
other_ch=`echo "$perm_other" | tr -d '-'`

permisos="u=$user_ch,g=$group_ch,o=$other_ch"

chmod $permisos "$archivo"

echo "permisos cambiados"

ls -l

```

10-Hacer un shell que lea las medidas de un terreno rectangular y determine el costo que tendrá cercar el terreno si considera que el precio por metro lineal de material es de 150 pesos y la mano de obra tiene un costo de 100 pesos. Se pide mostrar el perímetro del terreno, el total por concepto de mano de obra, el total por concepto de material y el costo total de la construcción.

```

#!/bin/bash

echo "ingrese lo largo del terreno"
read largo

echo "ingrese lo alto del terreno"
read alto

#calcular el perimetro

suma_lados=$((largo + alto))
perimetro=$((2 * $suma_lados))

costo_material=$((perimetro * 150))
costo_mano_obra=$((perimetro * 100))
costo_total=$((costo_material + costo_mano_obra))

echo "perimetro del terreno: $perimetro"
echo "costo del material: $costo_material"
echo "costo de la mano de obra: $costo_mano_obra"
echo "costo total de la obra: $costo_total"

```

```
└─$ ./queestion_10
ingrese lo largo del terreno
15
ingrese lo alto del terreno
5
perimetro del terrno: 40
costo del material: 6000
costo de la mano de obra: 4000
costo total de la obra: 10000
```

Se le pido los valores de las medidas tanto de largo como de ancho del terreno.

Usamos el operador `$()` para poder realizar el calculo dado a que todo lo que se ingresa en bash es de formato string hacemos uso del operador para hacer los cálculos.

Conclusión

Durante el desarrollo de la practica se pudieron desarrollar las habilidades del desarrollo de scritps con bash, algo muy importante en un sistema operativo, el scritp puede desarrollar tareas complejas, todo esta en un archivo que puede ejecutar tantas variables, funciones, estructuras selectivas, de repetición, etc. Asi como ejecutar comandos UNIX, con estos de pueden hacer proceso automáticos, muy importantes incluso cuando se desarrolla una app el hacer scripts para automatizar tareas que pueden ser repetitivas

Dificultades

El único inconveniente fue el tender mas o menos como es que trabaja este lenguaje, como funciona, al final es un lenguaje luego y toma tiempo en entender su sintaxis y su flujo de trabajo, pero mas allá no hubieras problemas graves, puede que el como era realizar operaciones matemáticas puesto que bash todo lo que ingresa es una cadena de caracteres.

Bibliografía:

<https://stackoverflow.com/questions/428109/extract-substring-in-bash>

<https://stackoverflow.com/questions/23564995/how-to-modify-a-global-variable-within-a-function-in-bash>

<https://phoenixnap.com/kb/bash-math>